Google Summer of Code

GA4GH-RegBot

Building a Clause-Precision RAG Assistant for Genomic Data Compliance

Project Title:	GA4GH-RegBot: Building a Clause-Precision RAG Assistant for Genomic Data Compliance
Programme:	Google Summer of Code 2026
Organisation:	GA4GH / Regulatory and Ethics Work Stream (REWS)
Name:	Ruiyang (Ringo) Chen
Email:	rc5663@nyu.edu
GitHub:	github.com/ringochen06
University:	New York University
Degree:	BA, Mathematics & Computer Science
Date:	March 2026

1 Project Abstract

GA4GH-RegBot is a lightweight, open-source Compliance Assistant that uses Retrieval-Augmented Generation to map genomic data-use consent letters against GA4GH regulatory frameworks. It delivers clause-precise, auditable compliance advice backed by two hard grounding controls — a chunk-ID allow-list and a token-overlap filter — so researchers get traceable answers rather than chatbot guesses, lowering the barrier to responsible data sharing.

2 Project Description

What Is Missing and Why It Matters

Genomic data sharing is bottlenecked not by willingness but by the complexity of cross-jurisdictional compliance. Researchers must manually cross-reference dense policy documents — the GA4GH Framework for Responsible Sharing, GDPR, the Consent Codes — against specific consent letter clauses. This is slow, error-prone, and a genuine barrier to science.

A generic LLM chatbot cannot solve this safely. Regulatory text is adversarial: "must" versus "should" changes an entire compliance landscape, and a hallucinated clause citation is worse than no answer at all. RegBot's core bet is that auditability must be enforced at the code level, not merely hoped for via prompting.

Technical Approach

The architecture rests on three layers, each addressing a known failure mode of naive RAG on legal text:

- **Phase 1 — Corpus and ingestion improvement:** Improve the existing ingestion pipeline to ingest real GA4GH, REWS, and regional regulatory documents. Refine chunk metadata (source, page, section, category, jurisdiction, chunk_id) and ensure the corpus is searchable and reproducible.
- **Phase 2 — Retrieval evaluation and improvement:** Build a mentor-reviewed evaluation set; evaluate and tune the hybrid retrieval pipeline (dense + BM25 + RRF). Add a re-ranking layer if it improves top-evidence quality.
- **Phase 3 — Grounded report generation and reliability:** Improve structured JSON reports so recommendations are better connected to retrieved evidence. Strengthen insufficient-evidence handling so the system can flag weak retrieval support and recommend human review. Two grounding controls already enforced in code:

- ◦ Chunk-ID allow-list: every cited chunk_id verified against the retrieved evidence set; hallucinated references trigger automatic retry.
- ◦ Token-overlap filter: each recommendation must meet a minimum token-recall threshold against cited chunk texts (REGBOT_MIN_TOKEN_OVERLAP). Results nested under grounding.overlap (kept_count, dropped_count, dropped_all, min_threshold).

What RegBot Brings to GA4GH

RegBot directly serves the GA4GH REWS mission: making responsible data sharing operationally tractable. A working open-source tool that gives researchers auditable, evidence-connected compliance answers can reduce the time-to-data for genomic studies globally.

Why I Chose This Project

My internship at Global Key Solutions gave me end-to-end experience building regulatory data pipelines for FDA and NMPA compliance. My double major in CS and Mathematics (NYU) gives me the formal background to reason about embedding-space geometry and retrieval quality. I have already built a working MVP of RegBot and contributed it upstream (PR #35). This is not a project I am learning about — it is one I am already executing.

3 Project Deliverables

May 1 – May 24, 2026 · Community Bonding

- Establish regular sync cadence with REWS mentor(s).
- Audit existing MVP codebase; agree on metadata schema and document scope.
- Identify real GA4GH, REWS, and regional regulatory documents for the corpus.
- Ensure CI, linting, and contributor tooling is accepted upstream.

Deliverable: Written design doc (metadata schema, document scope, evaluation plan) agreed with mentor.

May 25 · Coding Begins

Development environment is already working — the MVP runs locally and in CI from Day 1.

May 25 – Jun 14 · Phase 1: Corpus and Ingestion Improvement

Improve the existing ingestion pipeline and move from synthetic examples toward real GA4GH, REWS, and regional regulatory documents. Refine document metadata such as source, page, section, category, jurisdiction, and chunk ID where available. Add tests to make sure the ingested corpus is searchable, reproducible, and suitable for downstream retrieval.

Milestone: Real regulatory corpus ingested with improved metadata.

Jun 15 – Jul 5 · Phase 2: Retrieval Evaluation and Improvement

Build a small evaluation set of mentor-reviewed queries with expected supporting chunks. Evaluate the current hybrid retrieval pipeline and tune chunking, filtering, and retrieval parameters. If useful, add a re-ranking layer to improve the quality of top retrieved evidence.

Milestone: Evaluated and improved retrieval pipeline.

Jul 6 – Jul 10 · Midterm Evaluation

Present the real-corpus ingestion workflow, retrieval evaluation results, and current grounded report generation pipeline. Discuss remaining risks and agree on final-phase priorities with mentors.

Milestone: PASS: real corpus ingestion and evaluated retrieval.

Jul 11 – Aug 3 · Phase 3: Grounded Report Generation and Reliability

Improve the structured JSON report so recommendations are easier to review and better connected to retrieved evidence. Add more useful evidence information such as source document, page number, short quote or evidence span where available, and explanation of relevance. Strengthen insufficient-evidence behavior so the system can flag weak retrieval support and recommend human review instead of forcing an answer. Improve tests and documentation for the end-to-end workflow.

Milestone: End-to-end grounded regulatory navigation system.

Aug 4 – Aug 16 · Phase 4: UI Polish and Optional Interface Expansion

Improve the user-facing interface based on mentor feedback. Polish the existing Streamlit demo for clearer report review, evidence display, and download/export behavior. If time permits, explore a lightweight Next.js or similar web interface as a stretch goal for a more polished dashboard experience. Continue improving documentation, setup instructions, and demo materials.

Milestone: Polished demo interface and improved documentation.

Aug 17 – Aug 24 · Final Wrap-up

Clean up code, finalize documentation, prepare final demo materials, summarize design decisions and limitations, and submit the final work product with reproducible setup instructions, evaluation results, and a post-GSoC roadmap.

Milestone: RegBot final release candidate.

Post-GSoC

I intend to remain an active RegBot maintainer: expand the policy document corpus, explore a hosted demo for REWS stakeholders, and work toward integration with GA4GH data-sharing workflows.

4 Proposed Schedule

Period	Activities & Deliverables	Milestone
Mar 16 – Mar 31 Application	Finalise MVP contributions (PR #35); write and review proposal; build evaluation query set; seek mentor feedback.	Proposal submitted
Apr 1 – Apr 30 Acceptance Wait	Iterate MVP based on review feedback; add unit tests; read GA4GH Framework in full; draft design doc.	Codebase hardened
May 1 – May 24 Community Bonding	Weekly syncs with mentor; agree on metadata schema; identify real regulatory documents; confirm document scope.	Design doc approved
May 25 – Jun 14 Phase 1	Improve ingestion pipeline; ingest real GA4GH, REWS, and regional regulatory documents; refine metadata (source, page, section, category, jurisdiction, chunk_id); add ingestion tests.	Real regulatory corpus ingested with improved metadata
Jun 15 – Jul 5 Phase 2	Build mentor-reviewed evaluation set; evaluate and tune hybrid retrieval (dense + BM25 + RRF); add re-ranking layer if beneficial.	Evaluated and improved retrieval pipeline
Jul 6 – Jul 10 Midterm Evaluation	Present real-corpus ingestion workflow, retrieval evaluation results, and current grounded report generation. Agree on final-phase priorities.	PASS: real corpus ingestion and evaluated retrieval
Jul 11 – Aug 3 Phase 3	Improve JSON report: richer evidence info (source, page, evidence span, relevance). Strengthen insufficient-evidence handling. Improve end-to-end tests and docs.	End-to-end grounded regulatory navigation system

Period	Activities & Deliverables	Milestone
Aug 4 – Aug 16 Phase 4	Polish Streamlit demo (report review, evidence display, export). Explore Next.js interface as stretch goal. Improve docs and setup instructions.	Polished demo interface and improved documentation
Aug 17 – Aug 24 Final Wrap-up	Code cleanup, final documentation, demo materials, design decisions summary, post-GSoC roadmap.	RegBot final release candidate

Future Improvements

The following are proposed after the GSoC timeline, or as stretch goals if main deliverables complete early:

- **Multi-document cross-referencing:** A single DUL checked simultaneously against GDPR + GA4GH Framework + Consent Codes, with conflicting-clause detection.
- **Public benchmark dataset:** A curated, REWS-reviewed set of real DUL examples and ground-truth compliance verdicts.
- **pip-installable library:** Package RegBot with a stable public API for downstream integration.
- **FHIR / GA4GH API integration:** Explore hooking RegBot into GA4GH data-access request workflows for automated pre-screening.

Continued Involvement

I intend to stay involved with GA4GH and RegBot beyond the GSoC programme: maintain the codebase, triage issues, expand the policy corpus, and integrate with real data-sharing workflows.

5 Contribution History

Rather than describing future plans, the following contributions to GA4GH-RegBot are already merged or in active review — see PR #35.

Engineering and Collaboration Infrastructure

- **.gitignore and secrets hygiene:** Excludes .env files, virtual-environment layouts, and the local vector-store directory data/regbot_store/, eliminating accidental key or artefact commits.
- **GitHub Actions CI (tests):** Installs dependencies and runs the full unittest suite on every PR and push, catching regressions before merge.

- **GitHub Actions lint workflow:** A separate Ruff-based linting workflow enforcing code style on every PR, independent of the test suite.
- **pyproject.toml and requirements-dev.txt:** Project metadata, build config, and isolated dev dependencies for reproducible contributor environments.
- **.pre-commit-config.yaml:** Pre-commit hooks (Ruff, standard fixers) that catch issues locally before they reach CI.
- **CONTRIBUTING.md:** Full contributor guide covering setup, coding standards, testing instructions, and the PR workflow.
- **Unit tests:** Expanded from a smoke test to a suite covering utility functions, hybrid-fusion logic, grounding validation, token-overlap filtering behaviour, empty-PDF ingestion edge case, and end-to-end pipeline execution with mocked embeddings.

Core Product: RegBot MVP

- **Policy ingestion:** Supports PDF and plain text. Documents are chunked, embedded with SentenceTransformers, and written to a persistent Chroma store. A manifest.json records chunk text and metadata, serving as the source of truth for BM25 and citation verification. Ingestion raises a clear ValueError with OCR/encryption guidance when a PDF yields no extractable text.
- **Hybrid retrieval:** Dense vector search combined with BM25, merged via Reciprocal Rank Fusion (RRF). Supports metadata-category filtering for targeted retrieval within specific policy sections.
- **Compliance analysis with two-layer hard grounding:** Structured JSON compliance report (when OpenAI key present). Grounding enforced in code through two controls:
    - Chunk-ID allow-list: every cited chunk_id verified against retrieved evidence set; hallucinated references trigger automatic retry with an explicit allow-list.
    - Token-overlap filter: each recommendation checked for minimum token recall against cited chunk texts (REGBOT_MIN_TOKEN_OVERLAP). Results recorded under grounding.overlap (kept_count, dropped_count, dropped_all, min_threshold). Offline path skips overlap and records reason.
- **src/regbot/types.py:** Shared typed helpers for recommendation and grounding-related data structures.
- **Offline keyword fallback:** Gap analysis with chunk citations — works without any API key.
- **Operational hardening:** Chroma client anonymized-telemetry disabled by default (opt-in via REGBOT_CHROMA_ANONYMIZED_TELEMETRY); OpenAI max_retries configurable via REGBOT_OPENAI_MAX_RETRIES.

- **CLI + Streamlit UI:** python -m src.main with ingest, check, status, eval subcommands; Streamlit UI for policy upload, consent paste, JSON report download.
- **Reproducibility + docs:** Pinned dependencies; full README; examples/; run_demo.py; GA4GH-specific eval query file.

The ingestion pipeline, hybrid retrieval engine, dual hard-grounding controls, typed helpers, CLI, UI, contributor tooling, and CI infrastructure are not speculative — they exist and run today.

6 Testing Strategy

Current Test Coverage

- Unit tests for utility functions and hybrid RRF fusion logic.
- Grounding validation: chunk-ID allow-list enforcement and retry behaviour.
- Token-overlap filter: pass/fail/retry scenarios; disabled-filter behaviour.
- Ingestion edge cases: empty/unreadable PDF raises ValueError with guidance.
- End-to-end pipeline test with mocked embeddings and LLM responses.
- CI: GitHub Actions runs the full suite on every PR and push.

Summer Additions

- **Corpus ingestion tests:** Validate ingested real regulatory documents are correctly chunked, metadata-tagged, and searchable.
- **Retrieval benchmarks:** Precision@k and recall@k on a mentor-reviewed query set; automated regression gate in CI.
- **Report quality tests:** Verify evidence fields (source, page, evidence span) are correctly populated and grounding is enforced.
- **Integration tests:** Full pipeline from PDF ingest → query → JSON compliance report, covering both OpenAI-key and offline paths.

7 Appendix — Architecture Overview

Data flow from document ingestion to compliance report delivery:

```
INGEST PATH
  PDF / TXT  ->  [Parser: pypdf / PyMuPDF]  ->  Metadata-aware chunks
```

```
  {source, source_path, page, category, chunk_id,
   section*, jurisdiction*}  ->  ChromaDB  +  manifest.json
  (* improved in Phase 1)
```

```
QUERY PATH
  DUL Query  ->  Dense (SentenceTransformers)  +  BM25  ->  RRF Merge
             ->  [Re-rank layer if beneficial*]  ->  Top-k chunks
  (* Phase 2 deliverable)
```

```
GENERATION + GROUNDING
  LLM (OpenAI API)  ->  Draft JSON report
  -> [1] chunk_id allow-list check  -> retry if hallucinated
  -> [2] token-overlap filter        -> retry with repair prompt
  -> Final JSON: {recommendations, grounding: {ok, issues,
                  grounding_attempts, overlap: {kept_count, ...}}}
```

Sample Compliance Report Structure


```
{
  "query": "Does this consent permit secondary use for cancer research?",
  "recommendations": [
    {
      "text": "The consent permits use for health/medical research broadly.
               Cancer research falls within scope.",
      "evidence_chunk_ids": ["frs_2_1_p4", "frs_3_2_p7"]
    }
  ],
  "grounding": {
    "ok": true,
    "issues": [],
    "recommendation_count": 1,
    "grounding_attempts": 1,
    "overlap": {
      "kept_count": 1,
      "dropped_count": 0,
      "dropped_all": false,
      "min_threshold": 0.4
    }
  }
}
```


8 Major Challenges Foreseen

- **Real regulatory document heterogeneity:** GA4GH, REWS, and regional policy PDFs vary in structure, language, and quality. Mitigation: start with the most important documents, build targeted metadata extraction, and validate with REWS mentor.
- **Retrieval on specialised vocabulary:** Terms like "pseudonymisation" appear infrequently in general-purpose embeddings. Mitigation: BM25 for exact lexical match + evaluation-driven tuning.
- **Insufficient-evidence cases:** The system must fail gracefully when retrieval support is weak, recommending human review rather than hallucinating. Mitigation: explicit confidence thresholds and flag-rather-than-force logic in Phase 3.
- **Evaluation ground truth:** No existing labelled DUL dataset. Mitigation: build mentor-reviewed evaluation set during community bonding and Phase 2.
- **Open-source LLM instruction-following:** Smaller models are less reliable at structured JSON with strict citation format. Mitigation: few-shot prompting, JSON schema enforcement, existing grounding retry logic.

9 References

- GA4GH Framework for Responsible Sharing: https://www.ga4gh.org/genomic-data-toolkit/regulatory-ethics-toolkit/
- GA4GH RegBot repository: https://github.com/ga4gh/GA4GH-RegBot
- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401.
- Robertson & Zaragoza (2009). The Probabilistic Relevance Framework: BM25 and Beyond.
- Ragas: https://github.com/explodinggradients/ragas
- ChromaDB: https://docs.trychroma.com
- Sentence-Transformers: https://www.sbert.net

Ruiyang (Ringo) Chen

New York University · rc5663@nyu.edu

github.com/ringochen06 · ringochenry.com